Homework Reflection: Weeks 1-4

Roozbeh Khodayari

Boston University

M.S. in Data Science

Experimental Design and Causality

Homework reflection 1

1- In Coding Quiz 1, you are asked to find the distance of the farthest match in a set. Is this farthest match distance too far to be a meaningful match? How can you decide this?

The farthest match distance was 0.2102. To decide whether this distance was too large to be a meaningful match, I compared it with the distribution of all nearest-neighbor matching distances using a histogram, a boxplot, percentiles, and the IQR rule.

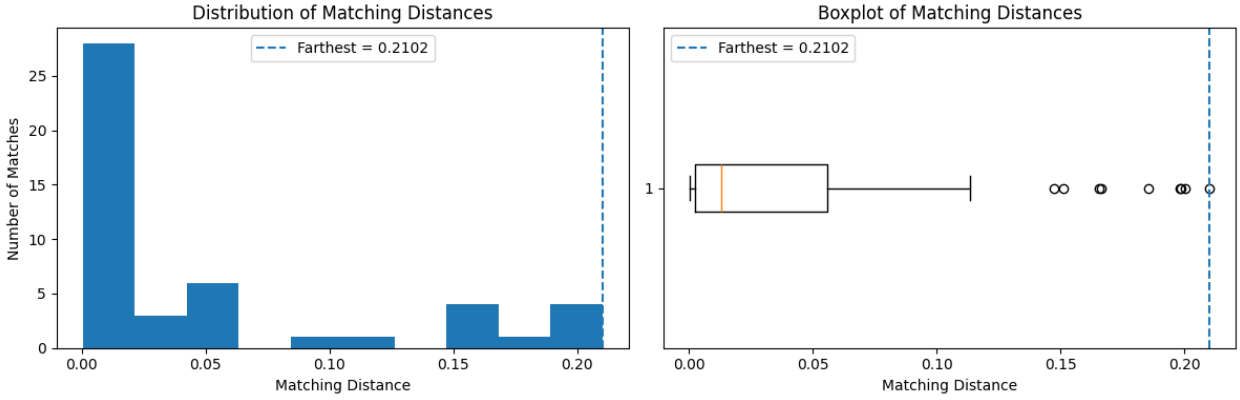
The median distance was only 0.0133, while the maximum distance was 0.2102, which is about 16 times larger than the median. The 90th and 95th percentiles were 0.1722 and 0.1985. The upper outlier limit based on the IQR rule was 0.1357, so the farthest distance was also above the IQR outlier limit. These results suggest that the farthest match is unusually distant compared with a typical match.

As an additional check, I used a scale-based caliper of 0.2 × SD(Z), where SD(Z) is the standard deviation of the matching variable Z. This gave a maximum acceptable distance of about 0.0580. The farthest distance of 0.2102 was well above this value, and 12 matches were outside the caliper. Together with the distribution and IQR results, this suggests that the farthest observation is too distant to be considered a meaningful match.

```
# import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.neighbors import NearestNeighbors
# Load the same data used in Coding Quiz 1
df2 = pd.read_csv('../data/homework_1.2.csv')
# Separate treatment and control groups
treatment = df2[df2["X"] == 1]
control = df2[df2["X"] == 0]
# Nearest-neighbor matching using Z
nn = NearestNeighbors(n_neighbors=1)
nn.fit(control[["Z"]])
```

```
distances, _ = nn.kneighbors(treatment[["Z"]])
match_distances = distances.flatten()
# Summary statistics
median_distance = np.median(match_distances)
p90 = np.percentile(match_distances, 90)
p95 = np.percentile(match_distances, 95)
q1 = np.percentile(match_distances, 25)
q3 = np.percentile(match_distances, 75)
iqr = q3 - q1
upper_iqr = q3 + 1.5 * iqr
max_distance = np.max(match_distances)
# Caliper based on 0.2 standard deviations of Z
sd_z = df2["Z"].std()
raw_caliper = 0.2 * sd_z
n_outside_caliper = np.sum(match_distances > raw_caliper)
print(f"Median distance: {median_distance:.4f}")
print(f"90th percentile: {p90:.4f}")
print(f"95th percentile: {p95:.4f}")
print(f"IQR: {iqr:.4f}")
print(f"Upper IQR outlier limit: {upper_iqr:.4f}")
print(f"Maximum distance: {max_distance:.4f}")
print(f"SD of Z: {sd_z:.4f}")
print(f"Raw caliper (0.2 x SD(Z)): {raw_caliper:.4f}")
print(f"Number of matches outside the caliper: {n_outside_caliper}")
# Histogram and boxplot side by side
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
# Histogram
axes[0].hist(match_distances, bins=10)
axes[0].axvline(max_distance, linestyle="--", label=f"Farthest =
{max_distance:.4f}")
axes[0].set_xlabel("Matching Distance")
axes[0].set_ylabel("Number of Matches")
axes[0].set_title("Distribution of Matching Distances")
axes[0].legend()
# Boxplot
axes[1].boxplot(match_distances, vert=False)
axes[1].axvline(max_distance, linestyle="--", label=f"Farthest =
{max_distance:.4f}")
axes[1].set_xlabel("Matching Distance")
axes[1].set_title("Boxplot of Matching Distances")
axes[1].legend()
plt.tight_layout()
plt.show()
```

```
Median distance: 0.0133
90th percentile: 0.1722
95th percentile: 0.1985
IQR: 0.0533
Upper IQR outlier limit: 0.1357
Maximum distance: 0.2102
SD of Z: 0.2898
Raw caliper (0.2 x SD(Z)): 0.0580
Number of matches outside the caliper: 12
```



2- In Coding Quiz 1, there are two approaches to matching:

A: Picking the best match X = 0 corresponding to each X = 1 using Z values.

B: Using radius_neighbors to pick all matches X = 0 within a distance of 0.2 of each X = 1.

Invent your own type of matching similar to 1 and 2 (or look one up on the internet), which has a different way to pick the matches in X = 0. Clearly explain the approach you invented or found.

Another approach is k-nearest-neighbor matching with a caliper. For each treatment observation, I would first find the three control observations with the closest values of Z.

I would then keep only the controls whose distance is within the caliper. In this example, I used 0.2 × SD(Z) as the caliper. This means that each treatment observation can be matched with up to three nearby controls, but any control that is too far away is removed. If none of the three controls are within the caliper, that treatment observation remains unmatched.

For each matched treatment observation, I use the average Y value of its accepted control matches as the comparison outcome. This differs from the first quiz method because it can use

more than one control, and it differs from radius matching because it uses at most three controls instead of using every control inside the radius.

```
# k-nearest-neighbor matching with a caliper
k = 3
caliper = 0.2 * df2["Z"].std()

nn_k = NearestNeighbors(n_neighbors=k)
nn_k.fit(control[["Z"]])

distances_k, indices_k = nn_k.kneighbors(treatment[["Z"]])

matched_control_means = []
matched_treatment_y = []

for i in range(len(treatment)):
    valid = distances_k[i] <= caliper
    valid_indices = indices_k[i][valid]

    if len(valid_indices) > 0:
        matched_control_means.append(control.iloc[valid_indices]["Y"].mean())
        matched_treatment_y.append(treatment.iloc[i]["Y"])

estimated_effect = np.mean(matched_treatment_y) - np.mean(matched_control_means)

print(f"Caliper: {caliper:.4f}")
print(f"Matched treatment observations: {len(matched_treatment_y)}")
print(f"Estimated treatment effect: {estimated_effect:.4f}")
```

```
Caliper: 0.0580
Matched treatment observations: 36
Estimated treatment effect: 0.5088
```

Homework reflection 2

1- Invent an example situation that would use fixed effects.

For example, I could study how work hours affect employee sales performance in different stores. Some stores may naturally have higher sales because they are in better locations, have more customers, or have different managers. If I ignore these differences, I may get the wrong idea about the effect of work hours. A fixed effects model can give each store its own baseline

sales level and then estimate the common effect of work hours on sales. This helps separate the store differences from the effect of work hours.

2- Write a Python program that performs a bootstrap simulation to find the variance in the mean of the Pareto distribution when different samples are taken. Explain what you had to do for this. As you make the full sample size bigger (for the same distribution), what happens to the variance of the mean of the samples? Does it stay about the same, get smaller, or get bigger?

```
np.random.seed(42)

def bootstrap_pareto_mean_variance(sample_size, shape_param=3,
num_bootstraps=1000):

    sample = np.random.pareto(a=shape_param, size=sample_size) + 1

    # 2. Perform Bootstrap Simulation
    bootstrap_means = []
    for _ in range(num_bootstraps):

        boot_sample = np.random.choice(sample, size=sample_size, replace=True)
        bootstrap_means.append(np.mean(boot_sample))

    # 3. Calculate the variance of the bootstrap sample means
    mean_variance = np.var(bootstrap_means, ddof=1)
    return mean_variance

# Evaluate variance across increasing sample sizes
sample_sizes = [100, 200, 500, 1000, 2000, 5000, 10000]

print("Sample Size (N) | Variance of Sample Mean")
print("-" * 40)
for N in sample_sizes:
    var_mean = bootstrap_pareto_mean_variance(sample_size=N)
    print(f"{N:15d} | {var_mean:.6f}")
```

```
Sample Size (N) | Variance of Sample Mean
----------------------------------------
            100 | 0.002921
            200 | 0.004311
            500 | 0.000946
           1000 | 0.000578
           2000 | 0.000280
           5000 | 0.000116
          10000 | 0.000109
```

I started by generating a sample from a Pareto distribution with a shape parameter of 3. For each sample size, I created many bootstrap samples by sampling with replacement from the original sample.

For every bootstrap sample, I calculated the mean. Then I used all of those bootstrap means to calculate their variance.

As the sample size increased, the variance of the sample means generally became smaller. It did not decrease at every step because of randomness, but the overall trend was clear. This suggests that larger samples give more stable estimates of the mean.

Homework reflection 3

1- In the event study in Coding Quiz 3, how would we go about testing for a change in the second derivative as well?

To test for a change in the second derivative, I would add a quadratic time term to the event-study model. For example, if the event occurs at time 50, I could use $(time - 50)^2$ and interact it with an after-event indicator, which is 0 before the event and 1 after the event.

This would let the model capture a change in curvature after the event. I would then check the coefficient and p-value of this interaction term. If the coefficient is significantly different from zero, it would suggest that the curvature changed after the event.

2- Create your own scenario that illustrates differences-in-differences. Describe the story behind the data and show whether there is a nonzero treatment effect.

One example of difference-in-differences would be two similar stores, where one store starts a new advertising campaign and the other store does not. The advertising campaign is the treatment.

Before the campaign, the treatment store has average sales of about 100 and the control store has average sales of about 80. After the campaign, their average sales increase to about 130 and 100. I simulated observations around these average values with some random variation.

The treatment store increased by about 30 units, while the control store increased by about 20 units. The difference-in-differences estimate is therefore approximately 30 - 20 = 10. Under the parallel trends assumption, the additional increase of about 10 units in the treatment store is the estimated treatment effect, so this example shows a nonzero treatment effect.

```
np.random.seed(42)
n = 300
control_before = np.random.normal(80, 5, n)
control_after = np.random.normal(100, 5, n)
treatment_before = np.random.normal(100, 5, n)
treatment_after = np.random.normal(130, 5, n)
df_did = pd.DataFrame({"sales":
np.concatenate([control_before,control_after,treatment_before,treatment_after]),
    "treated": np.repeat([0, 0, 1, 1], n),
    "after_campaign": np.repeat([0, 1, 0, 1], n)})
means = df_did.groupby(["treated", "after_campaign"])["sales"].mean()
control_change = means[0, 1] - means[0, 0]
treatment_change = means[1, 1] - means[1, 0]
did_effect = treatment_change - control_change

print(f"Control store change: {control_change:.2f}")
print(f"Treatment store change: {treatment_change:.2f}")
print(f"DiD treatment effect: {did_effect:.2f}")
```

```
Control store change: 19.92
Treatment store change: 30.08
DiD treatment effect: 10.16
```

Homework reflection 4

1- The Coding Quiz gives two options for instrumental variables. For the second item (dividing the range of W into multiple ranges), explain how you did it, show your code, and discuss any issues you encountered.

For the second IV method, I divided W into several smaller ranges and calculated the IV effect separately within each range. For each W range, I grouped the observations by Z and calculated the mean X and Y for Z = 0 and Z = 1. Then I calculated the effect as the change in Y divided by the change in X. Finally, I averaged the effects across the W ranges.

I initially tried pd.cut(), which divides the numerical range of W into equal-width intervals. However, when I increased the number of bins, some ranges had very few observations and the estimates became unstable. I therefore used pd.qcut() so that each range contained roughly the same number of observations.

I also tried several numbers of bins to check whether the result was sensitive to this choice. The estimates stayed close to 1.5; for example, using 10 ranges gave an estimated effect of about 1.51. The results were approximately 1.516, 1.509, 1.506, 1.503, 1.510, and 1.510 for 5, 10, 20, 30, 40, and 50 ranges, respectively. Since the estimates were very similar, the result appeared to be fairly stable across different numbers of W ranges.

```
df = pd.read_csv("data/homework_4.1.csv")
n_bins = [5, 10, 20, 30, 40, 50]
results = {}

for n in n_bins:

    df_n = df.copy()
    df_n["W_bin"] = pd.qcut(df_n["W"], q=n)
    effects = []
    for _, group in df_n.groupby("W_bin", observed=True):

        means = group.groupby("Z")[["X", "Y"]].mean()
```

```
        delta_y = means.loc[1, "Y"] - means.loc[0, "Y"]
        delta_x = means.loc[1, "X"] - means.loc[0, "X"]
        effects.append(delta_y / delta_x)
    results[n] = np.mean(effects)
results
```

```
{5: np.float64(1.5156437668194975),
 10: np.float64(1.508896371522371),
 20: np.float64(1.5055261177415453),
 30: np.float64(1.5028133965472348),
 40: np.float64(1.5103996612369919),
 50: np.float64(1.5096011303207189)}
```

2- Plot the college outcome (Y) vs. the test score (X) in a small range of test scores around 80. On the plot, compare it with the Y probability predicted by logistic regression. The ground truth Y value is 0 or 1; don't just plot 0 or 1 - that will make it unreadable. Find some way to make it look better than that.

```
# Load the data
df2a = pd.read_csv('./data/homework_4.2.a.csv')
# Small window around the cutoff
df_local = df2a[(df2a["X"] >= 75) & (df2a["X"] <= 85)].copy()
# RD features
df_local["centered_X"] = df_local["X"] - 80
df_local["treated"] = (df_local["X"] >= 80).astype(int)
df_local["interaction"] = df_local["centered_X"] * df_local["treated"]
# Logistic regression
X_logit = df_local[["centered_X", "treated", "interaction"]]
X_logit = sm.add_constant(X_logit)

logit_model = sm.Logit(df_local["Y"], X_logit).fit()
df_local["pred_prob"] = logit_model.predict(X_logit)
print(logit_model.summary())
```

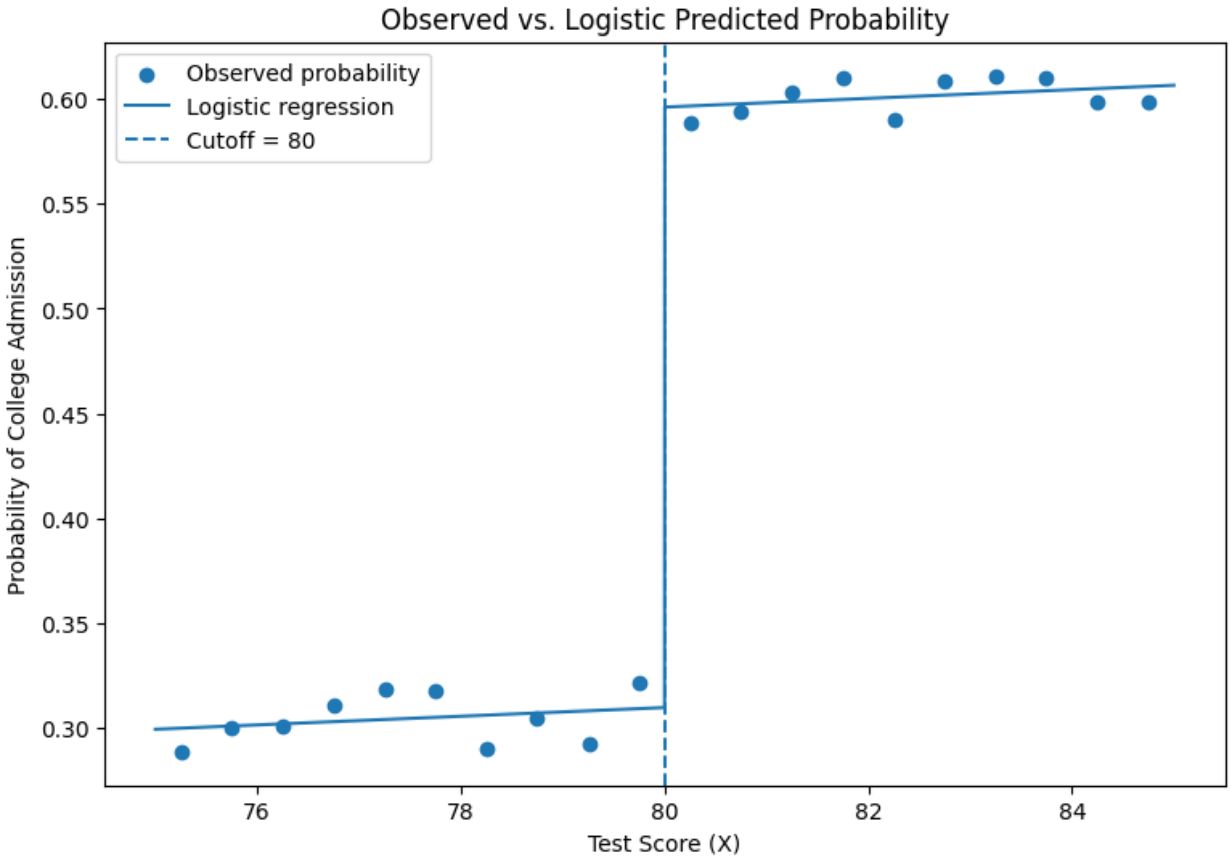
```
Optimization terminated successfully.
         Current function value: 0.643780
         Iterations 5
                           Logit Regression Results
==============================================================================
Dep. Variable:                      Y   No. Observations:                38276
Model:                          Logit   Df Residuals:                    38272
Method:                           MLE   Df Model:                            3
Date:                Thu, 24 Sep 2026   Pseudo R-squ.:                 0.06545
Time:                        00:51:58   Log-Likelihood:                -24641.
converged:                       True   LL-Null:                       -26367.
Covariance Type:            nonrobust   LLR p-value:                     0.000
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          -0.8011      0.031    -25.882      0.000      -0.862      -0.740
centered_X      0.0098      0.011      0.893      0.372      -0.012       0.031
treated         1.1906      0.042     28.076      0.000       1.107       1.274
interaction    -0.0011      0.015     -0.074      0.941      -0.030       0.028
==============================================================================
```

```
# bin the X variable into 20 equal-width bins and calculate the mean of X and the
mean of Y for each bin

df_local["X_bin"] = pd.cut(df_local["X"],bins=20)
observed = (df_local.groupby("X_bin", observed=True).agg(X_mean=("X",
"mean"),Y_prob=("Y", "mean")).reset_index())

# Plotting the observed and predicted probabilities
df_plot = df_local.sort_values("X")
plt.figure(figsize=(9, 6))

# Ground truth as observed probability
plt.scatter(observed["X_mean"],observed["Y_prob"],label="Observed probability")
# Logistic predicted probability
plt.plot(df_plot["X"],df_plot["pred_prob"],label="Logistic regression")
# Cutoff
plt.axvline(80,linestyle="--",label="Cutoff = 80")
plt.xlabel("Test Score (X)")
plt.ylabel("Probability of College Admission")
plt.title("Observed vs. Logistic Predicted Probability")
plt.legend()
plt.show()
```



Because Y is binary, plotting every observation directly at 0 or 1 would make the graph difficult to read. I divided the test scores between 75 and 85 into 20 small bins and calculated the average Y within each bin. Since Y is either 0 or 1, the average represents the observed probability of college admission.

I then fit a logistic regression using the centered test score, treatment indicator, and their interaction, and plotted the predicted probabilities using the original test-score values.

The observed probabilities were around 0.30 before the cutoff and around 0.60 after the cutoff. The logistic regression followed the same pattern and showed a clear jump at the cutoff of 80, while the slopes on either side were relatively flat.